UNIVERSITÉ HASSAN II DE CASABLANCA

ENSET MOHAMMEDIA | II BDCC

RAPPORT DE TRAVAUX PRATIQUES

TP 2

Docker Engine, Jenkins & CI/CD

Déploiement d'une application Nginx via une chaîne d'intégration et de livraison continues.

ÉTUDIANT	ENCADREMENT
Abdelali Saadali	Pr. Kamal EL GUEMMAT
Filière : II BDCC	Année universitaire : 2026/2027

Rapport d'état fondé sur les opérations et captures authentiques disponibles au 15 octobre 2026.

SECTION 00

Sommaire et synthèse

PARTIE	PAGES
1. Objectifs, outils et architecture	3
2. Application Nginx et dépôt GitHub	4
3. Intégration continue : Jenkins Freestyle	5–6
4. Pipelines, tests et Stage View	7–8
5. Déploiement et publication Docker Hub	9–10
6. Bilan, limites et conclusion	11–12

Synthèse des résultats

Le projet **tp4** a été créé sur GitHub et une page **Welcome BDCC** a été conteneurisée à l'aide de Nginx. Un serveur Jenkins persistant a exécuté des jobs Freestyle et des pipelines. Les captures attestent notamment des builds réussis **job1tp4 #1**, **job2tp4 #1**, **job2tp4v2 #2** et **job3tp4 #6**. Le conteneur **tp4-final** a servi une page web accessible sur le port **8082**.

Limite principale : le dépôt Docker Hub a bien été créé et Jenkins dispose d'une entrée Credentials masquée, mais aucune publication du tag **latest** n'est démontrée par les preuves disponibles. Le pipeline final possède cinq étapes, mais leurs noms et leur contenu diffèrent partiellement de l'énoncé.

Périmètre et méthode

Cette version présente les actions effectivement corroborées par les captures Jenkins, le terminal Codespaces, les pages GitHub et Docker Hub. Les exigences non démontrées sont consignées séparément, plutôt que présentées comme des réalisations validées.

SECTION 01

Objectifs et architecture technique

Chaîne étudiée : source Git → image Docker → Jenkins → conteneur Nginx

Objectifs pédagogiques

- Créer une page statique accessible via le serveur web Nginx dans un conteneur Docker.
- Versionner les sources sur GitHub et réaliser des opérations de construction et de vérification dans Jenkins.
- Découvrir les jobs Freestyle, les pipelines déclaratifs, le Stage View et le déploiement continu.

Environnement observé

COMPOSANT	CONFIGURATION OBSERVÉE
Espace de travail	GitHub Codespaces — /workspaces/tp2_guemmat
Dépôt applicatif	github.com/AbdelaliSaadali/tp4 — branche main
Serveur web	Nginx 1.27.4-alpine via Docker
Automatisation	Jenkins LTS, volume persistant jenkins_home
Réseau	Jenkins 8080 ; application finale Nginx 8082

Schéma simplifié

DÉVELOPPEMENT	CI / CD	EXÉCUTION
GitHub : tp4 / main	Jenkins : Freestyle + Pipeline	Docker Engine : Nginx
index.html + Dockerfile	Build / Test / Inspect / Deploy	tp4-final → port 8082

L'exécution se déroule dans un Codespace distant, sans installation de Docker ou Jenkins sur le Mac local. Les captures d'écran ont été prises à partir des interfaces du navigateur et de Jenkins.

SECTION 02

Application web et versionnement GitHub

Le dépôt public **AbdelaliSaadali/tp4** contient un **Dockerfile**, un **index.html** et un **README.md**. L'image repose sur **nginx:1.27.4-alpine**, copie la page dans **/usr/share/nginx/html/index.html** et expose le port 80. Ses métadonnées OCI identifient l'étudiant et l'établissement.

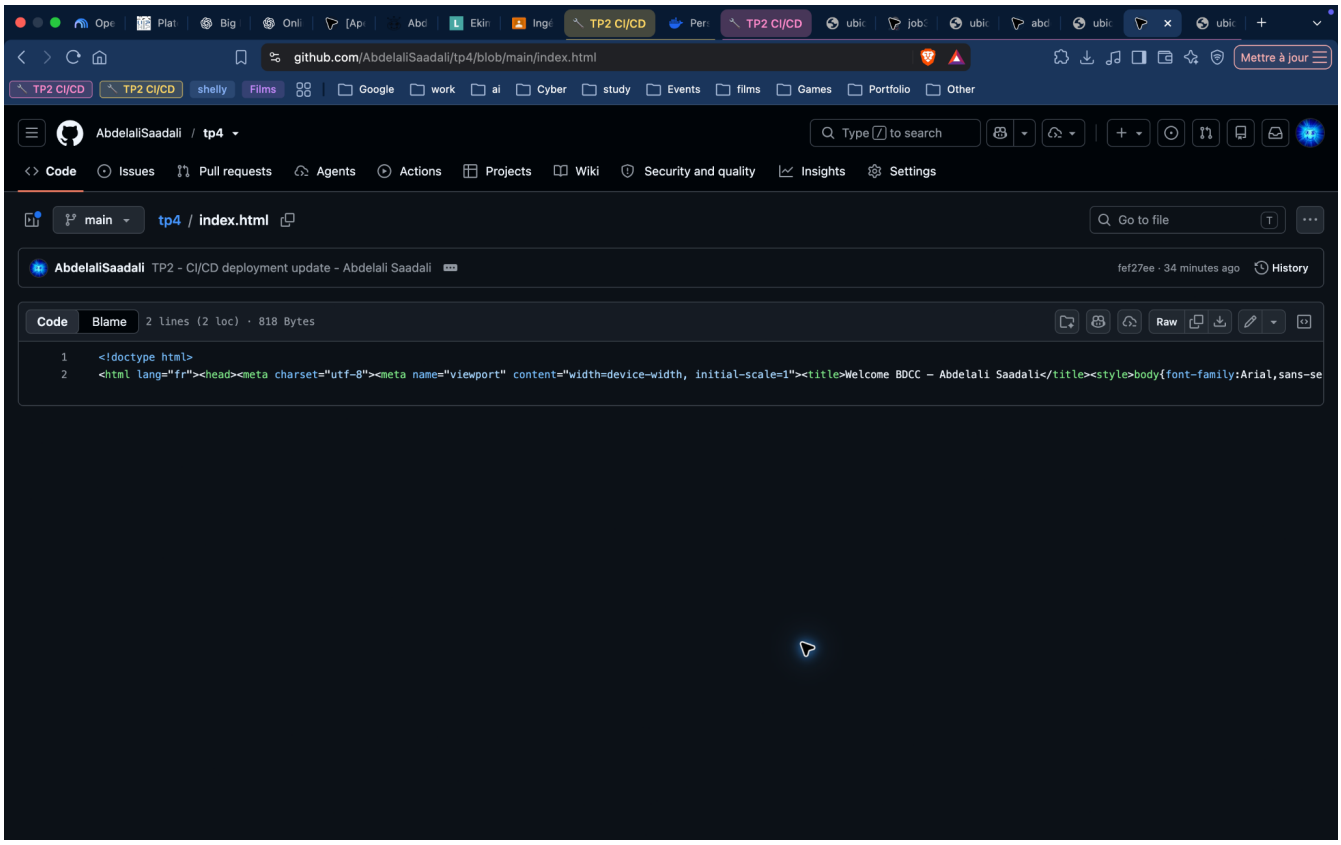


Figure 1 — Source index.html dans le dépôt GitHub tp4, avec l'identité Abdelali Saadali.

Le fichier source consulté sur la branche principale contient le titre « Welcome BDCC — Abdelali Saadali » et une mention « Version 2.0 — CI/CD Deployment Verified ». Cette révision du dépôt ne doit pas être confondue avec la version 1.0 visible dans le conteneur final capturé plus tard.

SECTION 03

Jenkins : exécution des jobs Freestyle

Le tableau de bord Jenkins montre quatre projets et leurs derniers builds réussis : **job1tp4 #1**, **job2tp4 #1**, **job2tp4v2 #2**, et **job3tp4 #6**. Les succès concernent les configurations exécutées à ces instants, sans prouver à eux seuls la publication sur Docker Hub.

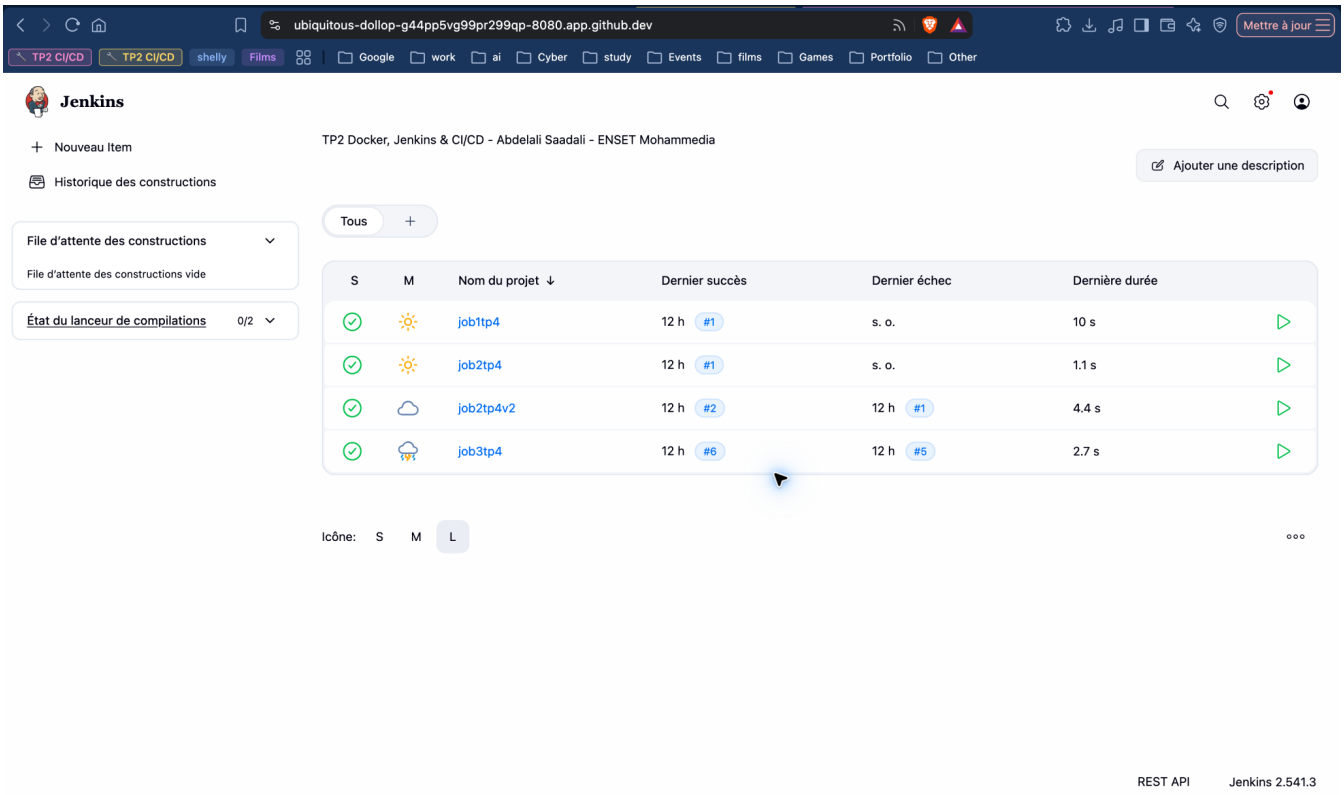


Figure 2 — Tableau de bord Jenkins identifiant Abdelali Saadali et les quatre jobs.

Le job **job1tp4** a assuré la récupération Git et une construction/test Docker réussis. Le job **job2tp4** a été déclaré réussi après un flux de construction, de vérification et de déploiement du conteneur, comme détaillé à la page suivante.

SECTION 03

Freestyle : preuve du déploiement job2tp4

Le build **job2tp4 #1** est marqué comme réussi par Jenkins. Il est associé au dépôt **https://github.com/AbdelaliSaadali/tp4.git** et à la révision **a57fc335…**. La configuration d'exécution décrite dans l'état du TP déploie le conteneur **tp4-deploy** sur le port **8083**.

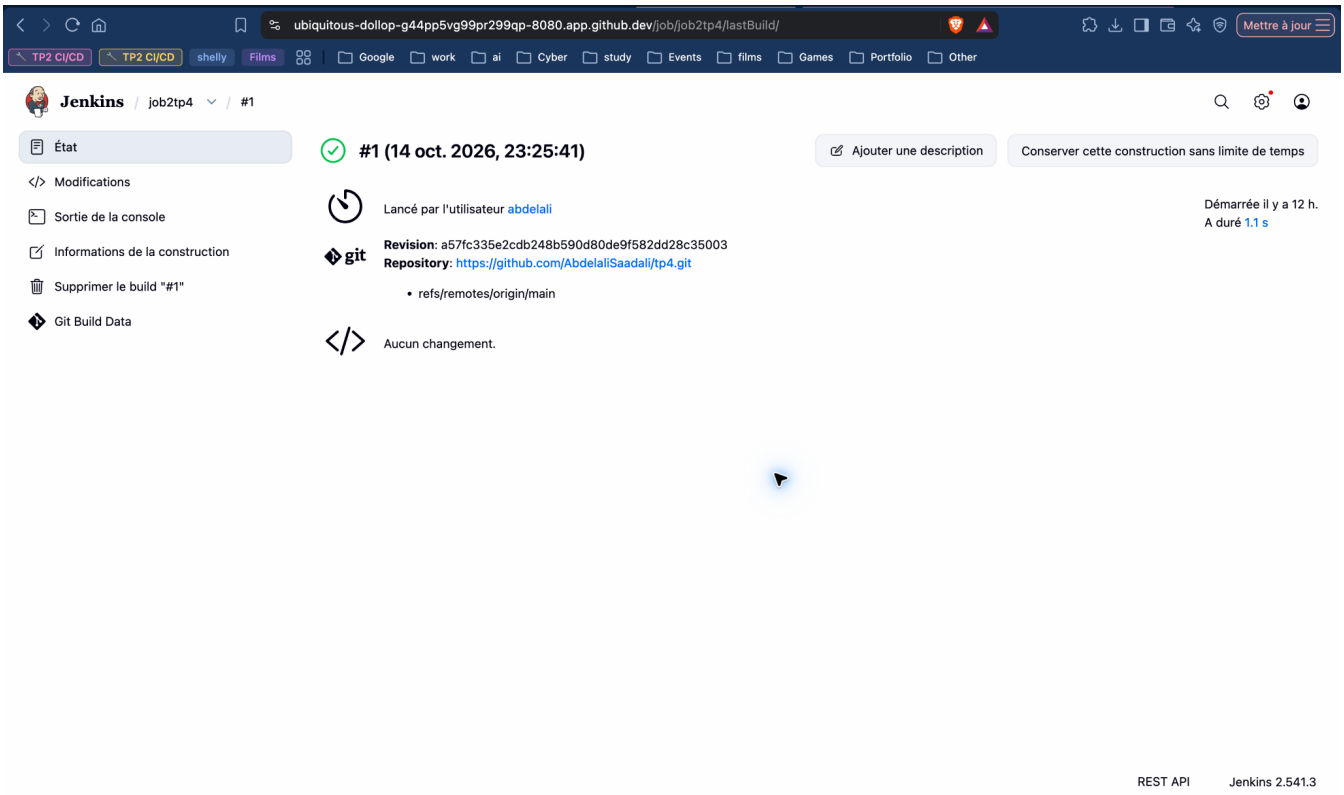


Figure 3 — Build Freestyle job2tp4 #1 : résultat SUCCESS et source Git utilisée.

Pour certifier la chaîne « build → publication → déploiement », les lignes de console doivent démontrer chaque opération. La capture ci-dessus prouve le résultat global du build, mais ne constitue pas à elle seule une preuve d'un push Docker Hub.

SECTION 04

Pipelines Jenkins et visualisation des stages

La vue **Stage View** du job **job3tp4** récapitule six exécutions, dont **#6** est entièrement verte. Cinq stages ont été effectivement affichés : **Checkout, Build, Test, Inspect, Deploy**. Les premiers essais présentent des échecs, puis un lancement abouti.

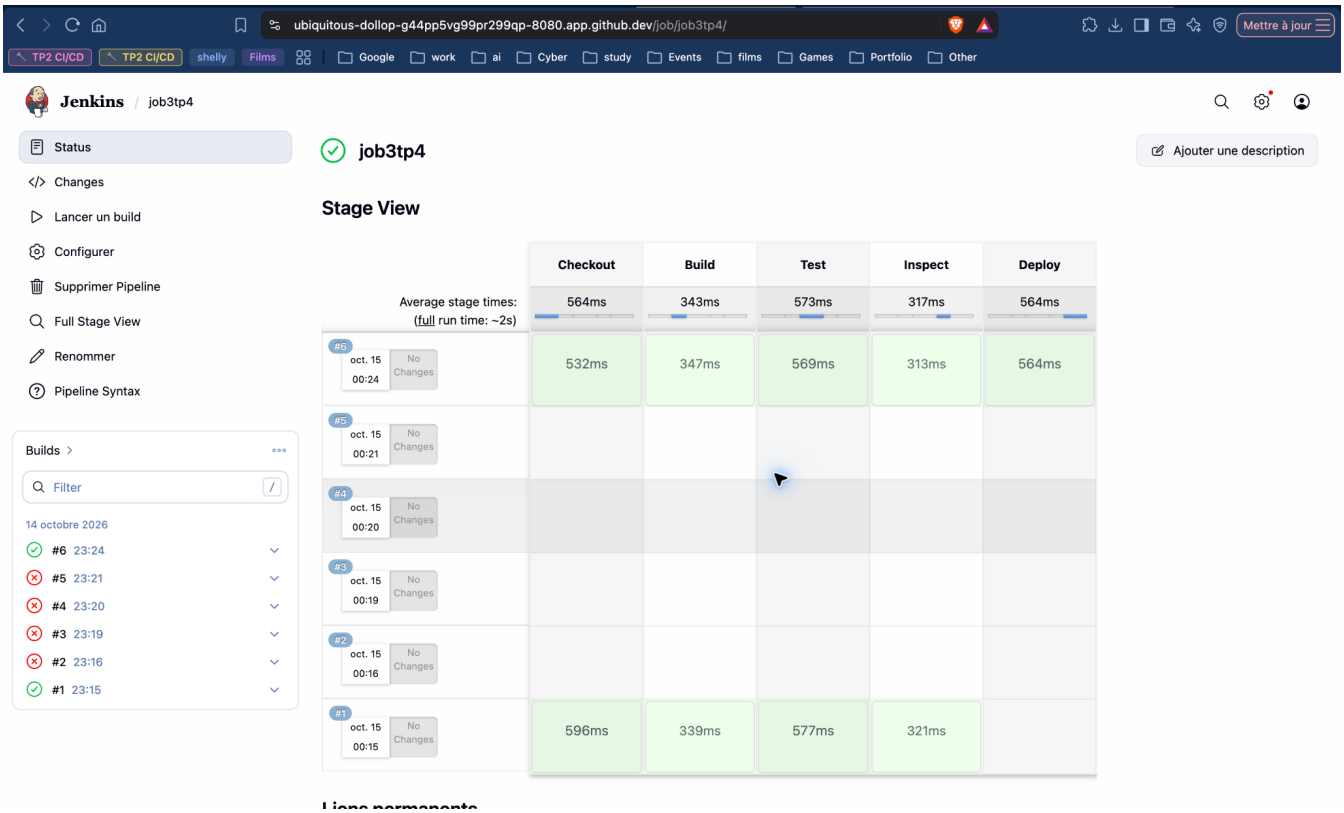


Figure 4 — Stage View de job3tp4 : build #6 réussi et historique des tentatives.

Écart avec l'énoncé : les cinq stages exigés portent les noms « Cloning Git », « Building image », « Test image », « Publish Image » et « deploy image ». Le stage actuel « Inspect » ne démontre pas la publication d'image.

SECTION 04

Construction de l’image Docker : sortie réelle

La console du stage **Build** montre l’exécution de **docker build -t tp4:latest .**. Les étapes visibles comprennent le téléchargement de l’image de base Nginx, l’ajout des labels OCI, la copie de **index.html** et l’exposition du port 80.

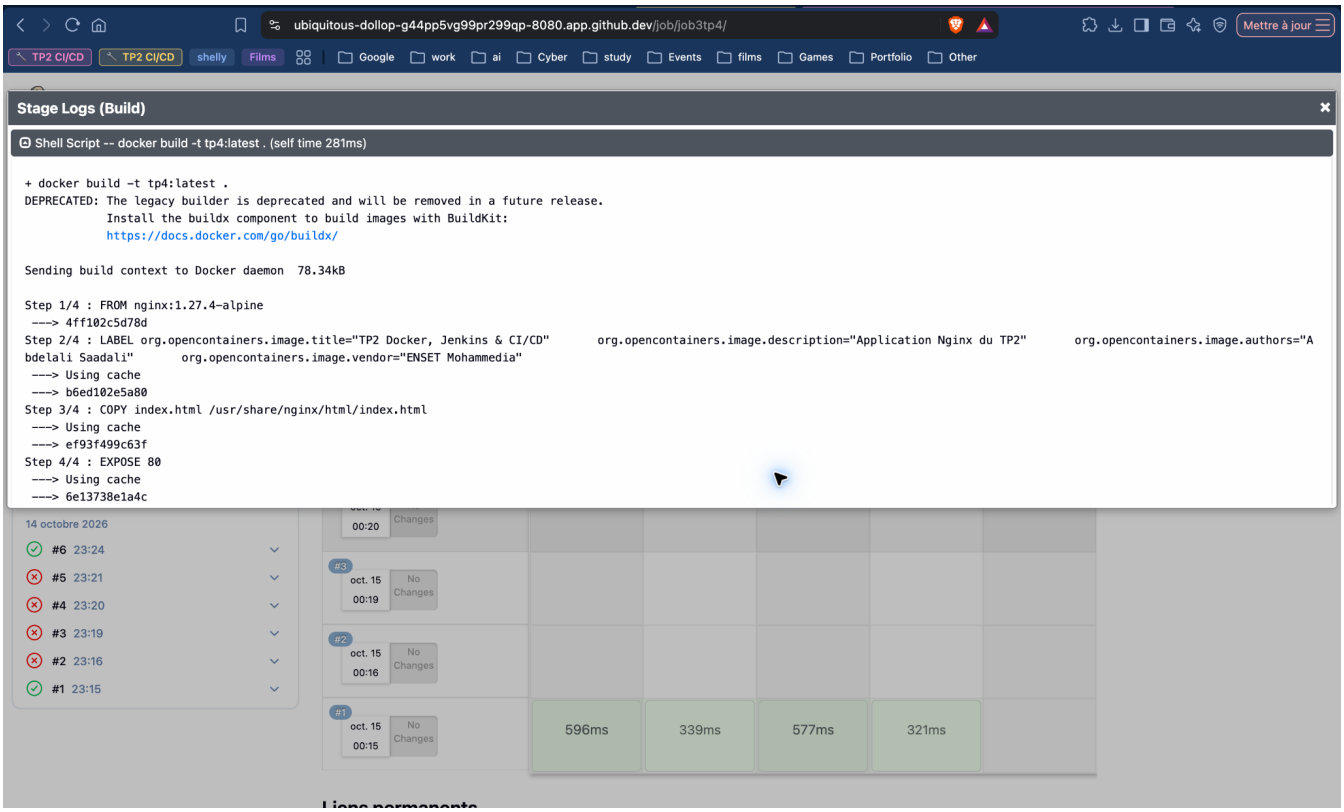


Figure 5 — Extrait authentique des logs du stage Build pour job3tp4.

Ce journal est une preuve directe du processus de construction de l’image. La mention de dépréciation du builder Docker ne constitue pas un échec du build. Le résultat global **job3tp4 #6** est attesté séparément par la vue Stage View.

SECTION 05

Déploiement : application Nginx accessible

Le conteneur **tp4-final** a été redémarré et la commande **curl http://localhost:8082** a renvoyé une page HTML valide. Le navigateur a ensuite affiché la page Nginx personnalisée au port 8082 : « Welcome BDCC » et « Réalisé par : Abdelali Saadali ».

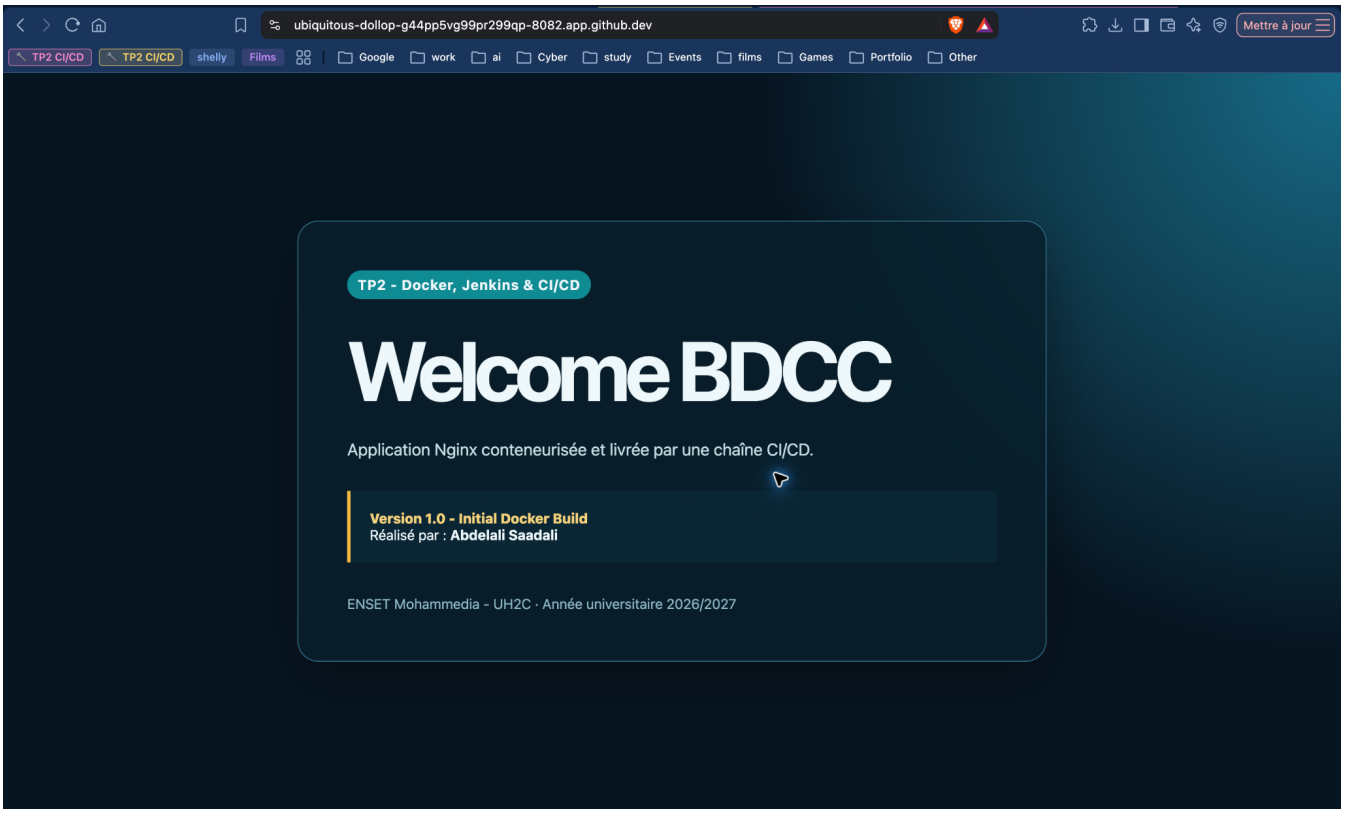


Figure 6 — Page web conteneurisée affichée depuis le port 8082 du Codespace.

Observation de version : la capture du conteneur affiche **Version 1.0 — Initial Docker Build**, alors que le fichier index.html de la branche GitHub principale contient une version 2.0. Il manque donc une preuve confirmant que la version 2.0 a été redéployée sur ce conteneur.

SECTION 05

Credentials Jenkins et registre Docker Hub

La configuration Jenkins contient une entrée globale de type identifiant/mot de passe nommée **dockerhub-tp2**, associée au compte Docker Hub **abdelalisaadali**. Le mot de passe n'est pas visible sur la capture.

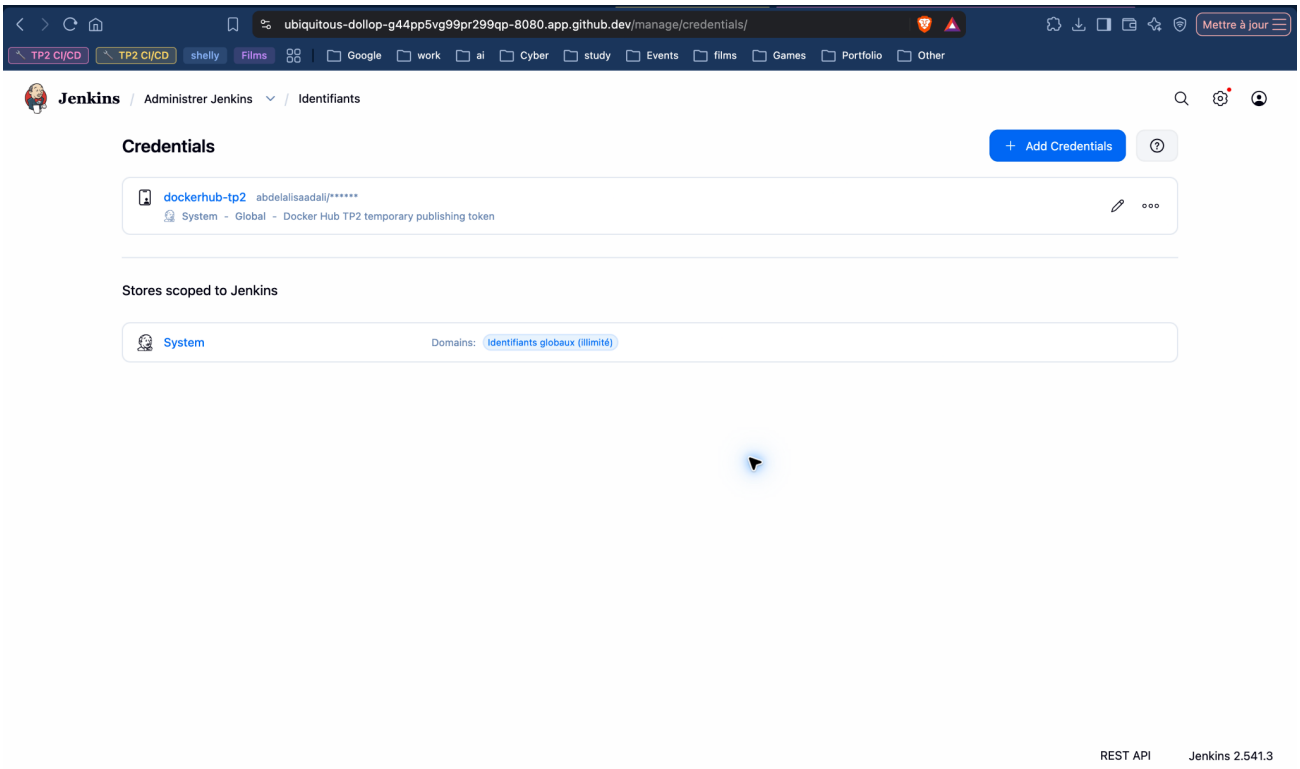


Figure 7 — Jenkins Credentials : entrée dockerhub-tp2 avec secret masqué.

Le dépôt Docker Hub **abdelalisaadali/tp4** a été créé, mais la capture du registre affiche une section « Tags » incomplète et ne présente aucun tag **latest**. L'existence du dépôt et celle d'un credential ne suffisent donc pas à établir une publication réussie.

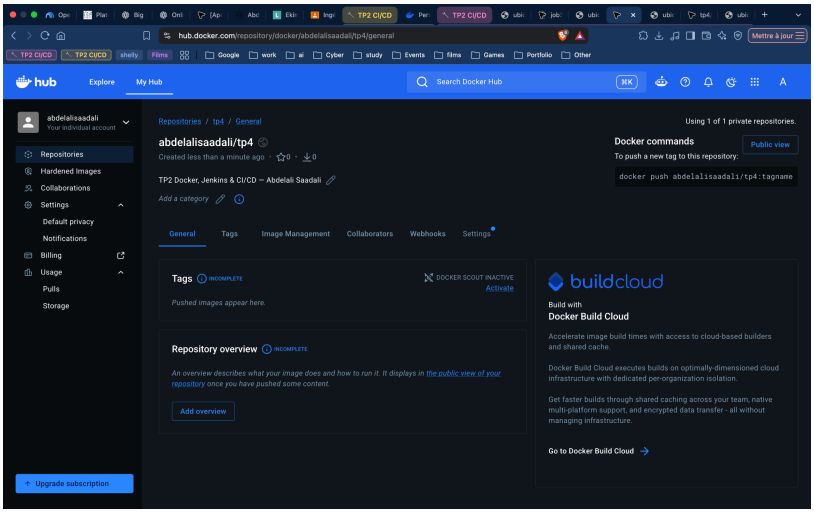


Figure 8 — Référentiel Docker Hub existant, sans tag publié visible lors de la capture.

SECTION 06

Bilan de conformité à l'énoncé

Le tableau ci-dessous distingue les preuves effectivement consultables des éléments non vérifiés. Cette distinction est essentielle pour un compte-rendu reproductible.

EXIGENCE DU TP2	ÉTAT / JUSTIFICATION
Jenkins installé et opérationnel	Vérifié — HTTP 200, interface et tableau de bord
Page « Welcome BDCC », Dockerfile Nginx, GitHub tp4	Vérifié — fichiers réels et application accessible
job1tp4 Freestyle / build	Vérifié — build #1 marqué SUCCESS
Docker Hub « latest » depuis Jenkins	Non vérifié — aucun push/tag visible
Modification HTML et nouveau build job1	Non documenté par les captures réunies
Suppression de job1tp4	Non effectué : job toujours visible
job2tp4 : build et déploiement	Partiellement vérifié — build #1 réussi et conteneur créé
job2tp4v2 en 3 stages	Build #2 réussi ; noms de stages requis non attestés
job3tp4 en 4 puis 5 stages	Vérifié en partie — Stage View #1 / #6 ; noms non conformes
Pipeline depuis un fichier jenkinsfile GitHub	Non vérifié — fichier absent du dépôt au contrôle
Vue Stage View après plusieurs builds	Vérifié — historique #1 à #6
Application déployée et consultable	Vérifié — port 8082 ; version affichée 1.0

Améliorations nécessaires

Ajouter un stage réel **Publish Image** avec Credentials Binding, publier et vérifier le tag **latest**, versionner un **jenkinsfile** au bon chemin, redéployer la dernière révision de la page, puis conserver les captures de vérification. Le job1tp4 ne doit être supprimé qu'après collecte complète de ses preuves.

SECTION 06

Conclusion et pièces justificatives

Ce TP a permis de mettre en place une chaîne locale de développement, construction, test et déploiement dans un Codespace Linux distant. L'association **GitHub + Docker + Jenkins + Nginx** a permis d'exécuter des jobs réels, de suivre les étapes d'un pipeline et de rendre accessible une application web personnalisée.

Les réussites observées concernent notamment les jobs Freestyle et la vue cinq stages du pipeline, ainsi que l'accès HTTP au conteneur. La publication dans Docker Hub, les stages nommés conformément à l'énoncé et le Jenkinsfile versionné restent des points à finaliser ou à documenter.

Références

- **Énoncé :** « TP 2 : Docker Engine, Jenkins, CI/CD », Pr. Kamal EL GUEMMAT, ENSET Mohammedia, année 2026/2027, pages 1–3.
- **Référentiel de code :** https://github.com/AbdelaliSaadali/tp4
- **Documentation Jenkins :** https://www.jenkins.io/
- **Documentation Docker :** https://docs.docker.com/

Index des preuves insérées

FIGURE	PREUVE
1	index.html — dépôt GitHub tp4
2	Vue d'ensemble des jobs Jenkins
3	Build Freestyle job2tp4 #1 réussi
4	Stage View job3tp4 #6 (cinq stages)
5	Console de construction Docker
6	Application Nginx en port 8082
7	Credentials Jenkins masqués
8	Dépôt Docker Hub — tag non vérifié

Note de sécurité : aucun jeton d'accès n'a été reproduit dans ce document. Le jeton Docker Hub apparu dans une capture séparée doit être révoqué après remise et remplacé en cas de poursuite des tests.